

Let $x \in \mathbb{R}^n$ and

$$\begin{aligned} x &= \frac{1}{n} \sum_{i=1}^n x_i \\ \text{minimize} \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \end{aligned}$$

Testing if the footnotes¹ work.

1 Introduction

Let $x \in \mathbb{R}^n$ and consider the average vector

$$\bar{x} = \frac{1}{n!} \sum_{P \in \mathcal{G}} Px.$$

Each coordinate of $\bar{x}$ is the mean of all entries of x , thus:

$$\bar{x} = \left(\frac{1}{n} \sum_{i=1}^n x_i \right) \mathbf{1} = \alpha \mathbf{1}.$$

2 Convexity and Invariance

Since f is convex and permutation-invariant,

$$f(\bar{x}) \leq f(x).$$

Therefore, if x is a minimizer, so is $\bar{x}$.

- 1. First item
- 2. Second item
- 3. Third item

$$\begin{aligned} x + y &= z \\ x - y &= w \\ x \cdot y &= v \end{aligned}$$

hello *fdsg*

```
\codelang{js}
\codetitle{Somethingtitle}
\codeshowlinenumbers{23}
\codehighlightlines{1-3,5}
\codehighlightwords{fp}
```

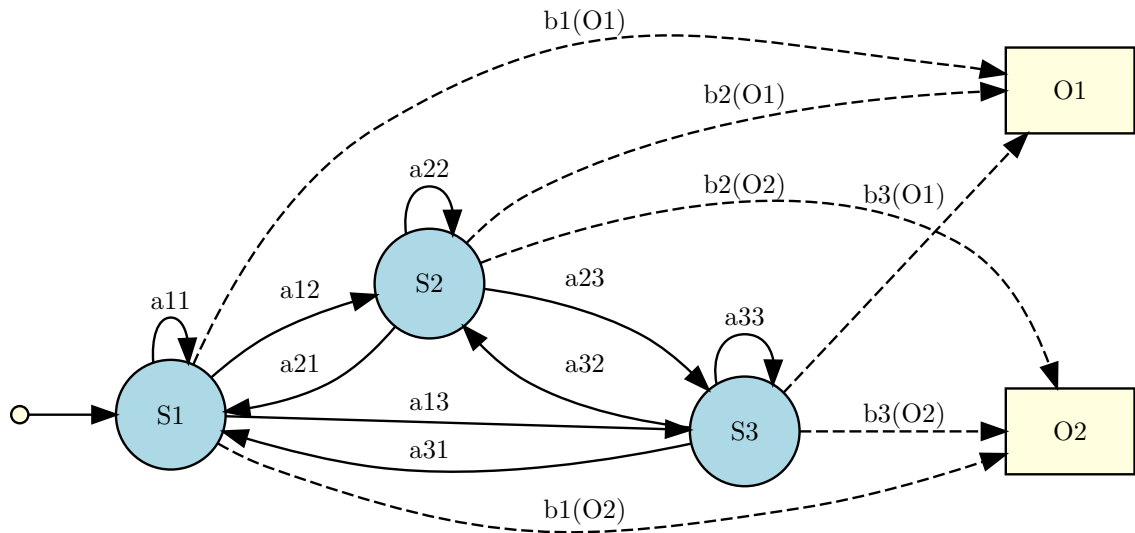
¹Do they work?

```

export function trimPathSuffix(fp: string): string {
  fp = clientSideSlug(fp)
  let [cleanPath, anchor] = fp.split("#", 2)
  anchor = anchor === undefined ? "" : "#" + anchor

  return cleanPath + anchor
}

```



2.0.1 See also

2.0.2 References

Algorithm 1 Complex-Test-Algorithm

```
1: Input:  $n, \gamma, \alpha$ 
2: for  $i = 1$  to  $n$  do
3:   if  $i \bmod 2 = 0$  then
4:     process even number  $i$ 
5:   else if  $i = 1$  then
6:     special case  $i = 1$ 
7:   else
8:     process odd number  $i$ 
9:   end if
10:  while  $i < n/2$  do
11:    increment  $i$ 
12:    break
13:  end while
14: end for
15: repeat
16:   compute something with  $x + y$ 
17: until  $x > y$ 
18: Switch-case not supported; use If/Else chain
19: if  $mode = 1$  then
20:   do mode 1
21: else if  $mode = 2$  then
22:   do mode 2
23: else
24:   default behavior
25: end if
26: Try/Catch not supported; describe in comments
27: risky operation
28: handle error if needed
29: cleanup task
30: Parallel/Async not supported; describe in comments
31: run parallel step
    return final value
```
